

Phinite — Support Chatbot Knowledge Rack

Condensed product documentation (full corpus summary)

2026-04-04

Contents

1	Phinite — Support Chatbot Knowledge Rack	2
1.1	1. Platform overview	2
1.2	2. What you can build	3
1.3	3. Workspace	3
1.4	4. Quick start (user journey)	3
1.5	5. Assistants	4
1.6	6. Assistant components (summary)	4
1.7	7. Flowgen Studio	4
1.8	8. Blocks and connections	5
1.9	9. Session variables (types)	5
1.10	10. RAG management	5
1.11	11. DevStudio (tools)	6
1.12	12. Triggers and intents	6
1.13	13. Builds and environments	6
1.14	14. Channels	7
1.15	15. Integrations Hub (catalog)	7
1.16	16. Observability and billing	8
1.17	17. Troubleshooting	8
1.18	18. FAQs (from documentation)	8
1.19	19. Keyboard shortcuts	8
1.20	20. Glossary (extended)	8
1.21	21. Answer patterns for support chatbot (optional routing hints)	9
1.22	22. Flowgen Studio interface (operational map)	9
1.23	23. Block-level reference (condensed)	9
1.24	24. Conversational assistant playbook	10
1.25	25. Email assistant playbook	10
1.26	26. Autonomous assistant playbook	10
1.27	27. Intents — creation and quality	10
1.28	28. Webhooks, API triggers, and security	11
1.29	29. Builds — configuration surface	11
1.30	30. User management and security	11
1.31	31. Observability depth (what to cite in tickets)	11
1.32	32. Environment-specific failures (runbook)	12
1.33	33. Integrations Hub — capability strings (support lookup)	12
1.34	34. Prebuilt DevStudio tools (family names)	13
1.35	35. Long-form FAQs (support)	13
1.36	36. Workflow creation methods (Flowgen)	13
1.37	37. Tool creation methods (DevStudio)	13
1.38	38. Intent testing and regression strategy	13
1.39	39. Observability filters (practical)	14
1.40	40. API usage patterns (generic)	14
1.41	41. Session log interpretation (support script)	14

1.42 42. Billing and cost optimization (guidance) 14

1.43 43. Access, compliance, and audit 15

1.44 44. Channel notes (per-channel pitfalls) 15

1.45 45. Data handling and PII (support positioning) 15

1.46 46. Incident response checklist (platform) 15

1.47 47. Advanced intent and trigger scenarios 16

1.48 48. Build vs flow publishing (terminology) 16

1.49 49. Copilot limits (user expectations) 16

1.50 50. Where to deepen (pointers for chatbot escalation) 16

1.51 51. Scenario-based Q&A (support retrieval) 16

1.52 52. Terminology crosswalk (synonyms users say) 18

1.53 53. Minimum viable triage (ticket fields) 18

1.54 54. Documentation map (full site paths) 18

1.55 55. PDF regeneration (maintainers) 19

1.56 56. Document control 19

1 Phinite — Support Chatbot Knowledge Rack

Purpose: Dense reference derived from Phinite product documentation for retrieval-augmented support. Plain text; no UI markup.

Scope: Platform overview, workspaces, assistants, Flowgen Studio, DevStudio, triggers and intents, builds and environments, channels, Integrations Hub, observability, troubleshooting, FAQs, and glossary.

1.1 1. Platform overview

Phinite is an AI automation platform to build, orchestrate, and deploy assistants and multi-agent workflows across channels, tools, and business processes (support, voice, email automation, reporting, autonomous jobs).

Core concepts

Concept	Meaning
Assistant	Conversational, email, or autonomous agent powered by one or more flows
Flow	Visual decision graph: nodes orchestrate logic, tools, and conversation
Node / block	Building block: prompt model, call tool, capture input, branch
Tool	Callable API, function, or system integration used by the assistant
Trigger	How an assistant is invoked (voice, webhook, schedule, email, API)
Intent	For conversational assistants: classification of user utterance mapped to a flow
Environment	Isolated stage: Dev, UAT, Production — separate configs, logs, caps
Session	One full execution of a flow; logged with inputs, decisions, outputs, metrics
Build	Versioned snapshot of an assistant: flows, intents, triggers, tools

Differentiators (messaging): Visual workflow builder (no-code and code paths), multi-agent orchestration (master/child agents), observability (sessions, tokens, timelines), RBAC and environment isolation, reusable tools and

templates.

Typical roles: Developers (tools, flows), Architects (logic, environments), Admins (access, billing), Testers (validation, logs), Analysts (sessions, usage).

1.2 2. What you can build

Conversational agents: Chat/voice; FAQs, scheduling, internal helpdesks. Capabilities: context, RAG, channels, RBAC, audit logs. Constraints: needs channels and knowledge setup; multi-turn behavior is flow-driven.

Email agents: Inbound/outbound automation; classify, respond, route, escalate. Integrations: Gmail, Outlook, SendGrid-class providers. Constraints: authenticated connectors; not a full mail client.

Automation / autonomous agents: Background and cross-system workflows; manager-worker patterns; schedules and webhooks. Constraints: orchestration layer (not raw infra automation); APIs must be configured and flows should be retry-safe.

Getting started path (high level): Create workspace → add users and roles → create assistant → design flows in Flowgen → deploy via environments and channels.

1.3 3. Workspace

A **workspace** is the collaboration boundary for assistants, tools, integrations, data sources (RAG), API keys, users, billing, and reports.

Home: Lists assistants, Copilot launcher, optional usage metrics.

Multi-workspace: Header switcher; Super Admins may create workspaces and assign Admins.

Sidebar areas (typical):

- **Assistants** — Open in Flowgen Studio, DevStudio, or Copilot.
- **Workspace tools** — Shared snippets, API connectors, utilities, reusable workflows across assistants.
- **Integrations** — Predefined (e.g. Jira, Gmail) and channel integrations (WhatsApp, Teams, Slack); authenticate and test before production.
- **Data sources / RAG** — Knowledge base: PDF, CSV, text, structured data, API-backed sources; organized in collections/folders.
- **API keys** — Workspace-level keys for programmatic access and “flow as a service”; revoke disables callers immediately.
- **Users** — Invites and roles (Super Admin, Admin, Developer, Tester, etc.).
- **Billing** — Plan, usage (tokens, compute, voice, API calls), invoices (Owners/Admins).
- **Reports** — Sessions, token/compute, assistant summaries; filter by environment.

Settings / docs / profile — Workspace settings, documentation link, user profile.

Best practice: One workspace per team or business unit for clarity and governance.

1.4 4. Quick start (user journey)

1. Verified account and workspace access; Copilot bar on home.
2. **Copilot:** Describe the assistant (e.g. FAQs + Zendesk tickets); Copilot creates assistant and opens Flowgen with generated nodes; tools auto-link via DevStudio where possible.
3. **Test:** Run in Copilot chat; inspect node-by-node execution in the timeline.
4. **Deploy:** Switch environment (e.g. Production), deploy/promote build, attach channel (web, chat, voice, API).

Common issues: Copilot unresponsive → refresh/connectivity. Blank canvas → create-assistant permissions. Test failures → DevStudio tool configuration. “Integration limit” → plan limits (docs mention Pro for multi-tool in quickstart).

1.5 5. Assistants

Definition: Central object in Phinite — combines flows, intents, triggers, tools, environments, builds, and Copilot into one deployable unit.

Types

Type	Best for	Triggers	Channels
Conversational	Real-time chat/voice	Intents, chat events	Web Chat, WhatsApp, Slack/Teams, Twilio, etc.
Email	Async mail workflows	Inbound email	Email
Autonomous	Background automation	Schedules, webhooks	N/A (system-side)

Lifecycle (conceptual): Trigger or intent fires → flow runs across blocks → tools act → response to user/system → logs under the environment.

Creating an assistant: Workspace → Create Assistant → choose type → Flowgen for graph → link trigger or intent → DevStudio tools → environment (Dev/UAT/Prod) → publish versioned build.

Copilot for assistants: Natural-language generation of flows, intents, triggers; uses workspace context and tools; refine before deploy.

1.6 6. Assistant components (summary)

- **Flows** — Visual workflows in Flowgen Studio; must connect to triggers and/or intents for entry.
- **Intents** — Utterance classification → mapped flow (conversational).
- **Triggers** — Webhooks, cron, email, events for autonomous flows.
- **Tools** — Built in DevStudio; attached to agent blocks.
- **Builds** — Frozen snapshots for release and rollback.
- **Environments** — Dev / UAT / Prod isolation for credentials and traffic.
- **Channels** — Where conversational assistants meet users (web, WhatsApp, Slack, Teams, Twilio voice, email).

1.7 7. Flowgen Studio

Purpose: Visual workflow builder for AI automations: canvas, collaboration, versioning, RBAC, environment deployment, audit.

Creation methods

- **Copilot:** Natural-language draft of structure.
- **Manual:** Drag blocks, edges, conditional routes, full control.

Capabilities

- **Canvas** (e.g. React Flow-style), live collaboration, version history and rollback.
- **Multi-agent:** Master agents (full tools/RAG/logic), child agents (reusable subgraphs), coordination and variable passing.
- **Logic:** Conditional routing via LLM decision variables; session variables across steps.

- **RAG:** Ground agents on workspace data sources/collections.
- **Debugging:** Execution logs, error boundaries, timing and success metrics.

Access: Workspace membership + role. Typical matrix: SuperAdmin/Admin — create/edit/publish flows and view logs; Developer — create/edit, publish often restricted to non-prod; Tester — test/view logs, no edit; Viewer — read-only. Exact UI may enforce Admin for production publish.

Prerequisites: Permissions, optional RAG setup, DevStudio tools and env vars, project access.

Publishing (summary): Save → validation → version notes → choose environment → publish. Only Admin/SuperAdmin for production in documented matrix.

Validation checklist: Required fields on all blocks; prompts complete; RAG linked; tools configured; variables mapped; conditional edges valid; Start and End present; no orphaned nodes.

Common failures: Missing required fields; invalid/orphan connections; tool auth or parameter errors — fix in Inspector, DevStudio, and re-test.

1.8 8. Blocks and connections

Block types

- **Start** — Entry; one outgoing path.
- **Master Agent** — Primary LLM unit: prompt, tools, RAG, variables.
- **Child Agent** — Embedded reusable agent subgraph.
- **Tool** — External API/tool execution node.
- **Sub-flow** — Invoke another flow (where enabled).
- **End** — Terminal node.

Connections: Handles connect nodes; edges define order and data flow. **Conditional edges** use decision variables to branch.

Agent block (four pillars): (1) Prompt — behavior and instructions. (2) RAG — collections from RAG management. (3) Tools — DevStudio tools and parameter mapping. (4) Variables — input and capture for downstream steps.

1.9 9. Session variables (types)

- **Capture variables** — Filled by the agent during run from natural-language descriptions.
- **Tool-returned capture variables** — From tool execution; custom tools can return `capture_variables` dict.
- **User variables** — Supplied in API payload for conversational/autonomous runs.
- **System variables** — e.g. `workflow_id`; `caller_number` on voice.
- **Predefined tool outputs** — Whole tool output object may be stored as a session variable; access fields with dot notation (e.g. `telegram.get_chat_member_count.count`).

1.10 10. RAG management

RAG grounds agents on enterprise knowledge.

1. **Data sources** — Register or connect sources (files, APIs, etc.).
2. **Collections** — Curate and organize items for retrieval.
3. **Referencing** — Attach collections per agent in the Inspector.

Practice: Keep sources relevant, organized, and updated; monitor which sources are used in logs.

1.11 11. DevStudio (tools)

Purpose: Author **tools** — versioned functions flows call (email, CRM, Slack, custom HTTP, DB, etc.).

Capabilities: Copilot-generated or manual code; typed parameters and schemas; draft/publish/rollback; test with sample payloads per environment.

Best practices: Validate inputs; defensive errors; log important steps; secrets in environment variables not code; meaningful version notes.

Types: Custom tools and prebuilt integrations (Gmail, Slack, Teams, Jira, MongoDB, custom API templates, etc. — see prebuilt docs).

Linking to agents: Open Agent block → Tools tab → select tools → map session variables to tool parameters → use capture variables downstream.

1.12 12. Triggers and intents

Intents (conversational): Name, description, training examples → train with prompt patterns → **map to flow** → test. Improve using log data. Related: prompt training, testing intents, mapping.

Triggers (autonomous / system): Start flows without chat — webhooks, cron, email events.

Webhook triggers: HTTP POST to platform webhook path with JSON payload; validate signatures; sanitize input; use idempotency and retries with backoff.

Event triggers: **Cron** (e.g. hourly sync) — note idempotency for repeats; **Email** — start on inbound messages.

Mapping: In trigger/intent UI, select target **flow version**; test in Dev/UAT before Prod.

Intent as API: On Edit Intent, enable trigger, pick integration funnel (e.g. Teams, Slack), choose flow. Platform exposes environment-specific **POST** URLs. Authenticate with **Bearer token** (workspace API key). Calling the API runs the intent and starts the conversation on the selected channel. Update build after intent changes.

1.13 13. Builds and environments

Build: Versioned package of flows and tools (and assistant dependencies) for deployment.

Lifecycle: Draft → validated → published (version notes) → deployed; promote Dev → UAT → Prod with approvals where required.

Environment purposes: **Development** — fast iteration; **UAT** — stakeholder QA; **Production** — live users. Separate credentials and monitoring per environment.

Assistant-level build steps (typical): Assistant → Build → Create Build → pick environment → validate flows/tools/triggers → deploy → monitor.

Best practices: Version notes every release; promote through stages; roll back to last stable if needed.

Common build failures: Missing required fields; tool tests failing in target env; missing version notes; wrong credentials for environment.

1.14 14. Channels

Channels connect assistants to users: **Web Chat**, **WhatsApp**, **Slack** / **Microsoft Teams**, **Twilio** (voice/SMS per setup), **Email**.

Integration pattern: Configure channel in workspace; test before production; combine with intents/triggers and builds.

1.15 15. Integrations Hub (catalog)

Phinite ships **many** production integrations; each has setup steps, credentials, and predefined tools in docs.

Phinite-native

- **API** — Generic REST client: any HTTP method; base URL; auth (API key, header, token, basic); custom headers; JSON/XML; rate limits and callbacks per configuration.
- **Voice** — Calls, TTS, call flows, audio (provider-dependent).
- **Chat (AI)** — LLM providers for generation, summary, translation, tool use.

Communication

- **Gmail** — Read/search/draft/send/reply; threads and labels.
- **Slack** — Post, channels, threads, history, approvals.
- **Microsoft Teams** — Messages, approvals, Graph permissions, notifications.
- **WhatsApp** — Templates, media, inbound, provider compliance.
- **Telegram** — Bot messages, commands, groups.
- **Email (SMTP)** — Transactional mail, templates, attachments, CC/BCC, TLS.
- **Twilio** — SMS, WhatsApp, voice, delivery status, numbers.

Data and storage

- **Google BigQuery** — SQL, tables, export, analytics assistants.
- **Google Sheets** — Read/write/append, light reporting.
- **Google Drive** — Upload/download/search, permissions, folders.
- **MongoDB** — CRUD, aggregations, indexes.

Developer tools

- **GitHub** — Repos, issues, PRs, comments, automation.
- **Jira** — Issues, transitions, comments, JQL.

Productivity

- **Google Calendar** — Events, availability, invites, scheduling automation.
- **Notion** — Pages, databases, search, blocks.

Sales

- **Salesforce** — Leads, accounts, opportunities, SOQL, sync workflows.

AI / web

- **Brave Search** — Web/news/image search for assistants.
- **Firecrawl** — Scrape/crawl, structured extract, change monitoring, screenshots/PDFs.

Support answers should steer users to the specific integration page for field-level setup.

1.16 16. Observability and billing

Usage metrics: Sessions, tokens, telephony usage (where applicable).

Logs: Timeline of execution; decision points and exceptions; variable capture traces. **Practices:** Filter by flow version and environment; trace failures to tool calls; export for audits.

Filtering / search: Slice by time, user, assistant, environment.

Billing: Plan, allocation, cost visibility (ties to token/compute/voice/API usage).

1.17 17. Troubleshooting

HTTP errors (typical):

- **401** — Token or API key; clock/skew; wrong environment key.
- **403** — Role or workspace permission.
- **404** — Wrong resource ID or deleted assistant/flow/tool.
- **422** — Validation on payload or tool parameters.
- **500** — Server-side; check logs and retry.

Debug sequence: Reproduce with minimal input → session logs → tool/flow config → environment credentials → version mismatch (build vs channel).

Support bundle: Steps to reproduce, inputs, expected vs actual, error text, environment, assistant/build IDs, timestamps from logs.

Contact (from docs — replace with your org’s channels if different): Email and portal placeholders in source docs; always include repro steps.

1.18 18. FAQs (from documentation)

- **Reuse flows across assistants?** Yes — map multiple assistants or intents to the same flow where allowed.
 - **Coding required?** Not for all flow work; custom tools often need code or API knowledge.
 - **Who can publish?** Admin or SuperAdmin (per role matrix).
 - **Different roles per workspace?** Yes.
 - **Tool failing?** Parameters, credentials, environment, logs.
 - **Intent misfiring?** More/better examples, retrain, review logs.
-

1.19 19. Keyboard shortcuts

Flowgen: Cmd/Ctrl+click multi-select; Delete removes edge/node; Space+drag pan; Cmd/Ctrl +/- zoom.

DevStudio: Cmd/Ctrl+Enter run tool test; Cmd/Ctrl+S save.

1.20 20. Glossary (extended)

Term	Definition
Assistant	Deployable AI capability: conversational, email, or autonomous
Flow	Directed graph of blocks representing automation
Agent block	LLM step with prompt, optional RAG, tools, variables

Term	Definition
Tool	Executable integration or function from DevStudio
RAG	Retrieval-augmented generation from collections
Intent	Utterance class → route to flow (chat)
Trigger	External or scheduled start of a flow
Webhook	HTTP callback trigger
Build	Versioned deployable snapshot
Environment	Dev / UAT / Prod runtime boundary
Session	Single run: chat, call, or job execution
Token	LLM usage unit for cost and metrics
Copilot	NL assistant for building/editing flows and tools
Integration funnel	Channel selection for intent-as-API and similar flows
Decision variable	Value used to choose conditional edges
Capture variable	Value extracted or returned for later blocks

1.21 21. Answer patterns for support chatbot (optional routing hints)

- **“Cannot publish to production”** → Role must be Admin/SuperAdmin; confirm environment; complete validation.
- **“Tool works in Dev, fails in Prod”** → Separate credentials; stricter networking; confirm build promoted; retest with same payload.
- **“RAG not used”** → Collections attached in agent; data indexed; query matches content; check logs for retrieval.
- **“Webhook not firing”** → URL and method; auth; idempotency; payload schema; logs for trigger execution.
- **“Intent API 401”** → Workspace API key in **Authorization: Bearer**; correct environment URL.

1.22 22. Flowgen Studio interface (operational map)

Understanding where configuration lives reduces support time.

- **Canvas** — Spatial layout of the workflow; pan/zoom; multi-select; delete edges/nodes. Execution is defined by graph topology plus conditional edges.
- **Block library** — Palette of Start, Master Agent, Child Agent, Tool, Sub-flow (if enabled), End. Drag onto canvas then wire handles.
- **Inspector** — Context for the selected block: for agents, tabs typically cover Details (name, core prompt `task_prompt`), RAG attachments, Tools enablement, Variables (input/capture/decision). For tool blocks: tool id, parameter mapping from session variables.
- **Variables panel** — Central list of session variables: creation, renaming, and references used in prompts and mappings. Align names across capture sites and downstream blocks to avoid “undefined variable” class issues in logs.

Collaboration: Multiple editors may see live updates; still coordinate on publish ownership to avoid conflicting drafts.

1.23 23. Block-level reference (condensed)

Start — Single entry; ensure exactly one coherent entry path into the graph for the mapped intent/trigger.

Master Agent (task node) — Fields: human-readable **Name**; **Prompt** (`task_prompt`) for role, policy, and output shape; **Tools** from workspace; **RAG** collections; **Input variables** fed from user payload or upstream; **Capture variables** for facts the model should extract; **Decision variables** that drive labeled outgoing edges for branching.

Child Agent — Encapsulated subgraph behavior for reuse; own prompt and variables; use when the same sub-logic appears in multiple flows.

Tool block (api node) — Select tool; map session variables into **Inputs**; outputs feed capture variables or predefined tool variable objects for dot-notation access.

End — Terminates the run; ensure all meaningful paths reach an End or intentional loop exit.

1.24 24. Conversational assistant playbook

Purpose: Real-time chat and voice with intent routing.

Steps (documentation order): (1) Define **intents** with names, descriptions, and training utterances. (2) **Train** prompts with diverse realistic phrasing; add edge cases; avoid narrow overfitting. (3) **Map** each intent to a specific flow version. (4) **Connect channels** — Web Chat, WhatsApp, Slack/Teams, Twilio voice — with correct OAuth/API credentials per environment. (5) **Observe** — Use logs to see which intent fired and where the flow branched; refine examples when misclassification appears.

Tip: Logs are the feedback loop for intent quality — mine failure cases into new training lines.

1.25 25. Email assistant playbook

Purpose: Inbound classification, routing, replies, ticketing side-effects.

Setup chain: Connect **email channel** → build **flows** that parse content → attach **tools** (CRM, helpdesk, SMTP reply). Use **capture variables** early to extract sender, subject, thread id, order numbers, and intent classifiers for deterministic tool calls.

Constraints: Depends on authenticated mailbox connectors; compliance and retention follow customer policy — document that Phinite automates flows, not archival/legal hold by itself.

1.26 26. Autonomous assistant playbook

Purpose: Schedules and external events without a live user turn.

Triggers: **Webhooks** for push events; **Cron** for periodic sync; **Email** triggers for mail-driven starts.

Best practices: **Idempotency** keys on external writes; **retries** with exponential backoff; **alerting** on repeated failures; **audit** via observability logs for each scheduled run.

Distinction: Good for operations, enrichment, reporting pipelines; not a substitute for low-level infrastructure automation unless tools expose that surface.

1.27 27. Intents — creation and quality

Creation flow: Create intent → name + description → add **only realistic user utterances** as examples → train and review confidence → map to a flow → test in intent tester.

Quality bar: Aim for strong **precision and recall** on held-out phrases; include negative or near-miss examples if the model confuses two intents.

Prompt training patterns: Pair utterance → intent label; include variations (“Where is my order?”, “Track shipment”, “Order status please”) mapped to one intent such as `order_status`.

Iteration: Export misroutes from **session logs** and add corrective examples; redeploy build after intent changes.

1.28 28. Webhooks, API triggers, and security

Webhook endpoint: Documented pattern `POST /api/triggers/webhook/:id` with JSON body (example keys like `event`, `data`). Response may include `status` and `triggerId`. Always validate signatures where provided, sanitize payloads, and reject unexpected types.

Generic API authentication: Authorization: Bearer YOUR_API_KEY on REST calls. Never embed keys in client-side code. Rotate keys from workspace settings when staff changes.

Intent-as-API: Bearer token required; environment-specific URL; triggers flow and starts channel conversation — verify integration funnel matches the customer’s expected channel (Teams vs Slack vs web).

1.29 29. Builds — configuration surface

Version notes — Required for audit; should describe user-visible changes, risk, and rollback implications.

Target environments — Choose Dev/UAT/Prod explicitly; production may need approval per org policy.

Inclusion — Confirm which flows and tool versions are part of the build artifact.

Per-environment settings — Endpoints, API keys, feature flags, quotas and rate limits. **Never hardcode secrets** in flow JSON; use environment variables.

Publishing flows and tools (combined checklist): Validate lint/tests → add version notes → publish → deploy to target env → smoke test → monitor metrics.

1.30 30. User management and security

Invites: Workspace → Users → Invite → email + role → pending state supports resend/revoke.

Lifecycle: Change roles when duties shift; remove users who offboard; audit access periodically.

Access control: Permissions are **per workspace and project**; least privilege; separate operator access across Dev/UAT/Prod where possible.

Authentication: Session tokens expire; invalid token leads to login — users should retry login after password resets or token revocation.

1.31 31. Observability depth (what to cite in tickets)

Usage metrics: Session counts; token usage (LLM and often STT/TTS where voice); telephony minutes or events.

Practices: Alert on spikes; correlate anomalies with release timestamps; break down by assistant and environment to isolate bad deploys.

Billing dashboard: Costs by assistant and environment; token/telephony/runtime splits; trends. **Practices:** Budgets and alerts; tie cost jumps to new tools or model choices; optimize expensive tool chatter.

Logs — three lenses: (1) **Timeline** — ordered steps with latency. (2) **Decision joints / exceptions** — branch choices and failures. (3) **Variables** — captured values at each step — essential for “why was this empty?” cases.

Filtering: Narrow by time range, user, assistant, environment, flow version — always ask for these in support intake.

1.32 32. Environment-specific failures (runbook)

Symptoms: Passes in Dev only; auth errors in one env; intermittent timeouts in Prod.

Checklist: Confirm **different credentials** per environment; **base URLs** for external APIs; **feature flags**; **egress/firewall** rules for SaaS APIs; **IP allowlists** for webhooks; **rate limits** that stricter tiers hit only in Prod.

Safe logging: Log variable *names* and non-secret metadata; never paste secrets into tickets.

1.33 33. Integrations Hub — capability strings (support lookup)

Use these phrases when users ask “what can X do?” without opening each page.

Integration	Capability summary
Google Calendar	Create/update/query events, availability, invites, scheduling automation.
Firecrawl	Scrape to markdown, crawl sites, structured extract, change monitoring, screenshots/PDFs.
Brave Search	Web/news/image search with snippets for research steps.
Salesforce	Leads, accounts, opportunities, SOQL, CRM workflow sync.
Telegram	Bot send/receive, commands, groups, notifications.
Notion	Pages/databases, search, append blocks, knowledge automation.
GitHub	Repos, issues, PRs, comments, developer workflow automation.
Gmail	Read/search/draft/send/reply, threads, labels.
BigQuery	SQL queries, tables, exports, analytics grounding.
Google Sheets	Read/write/append rows, light formatting, reporting.
Google Drive	Upload/download/search, permissions, folders.
MongoDB	CRUD, aggregations, indexes, flexible document data.
Chat (AI)	LLM generation, summarization, translation, tool-calling patterns.
Voice	Calls, TTS, recording, provider-specific call flows.
API (generic)	REST client with auth headers, retries, arbitrary JSON/XML.
Jira	Create/update/search issues, transitions, JQL.
Microsoft Teams	Messages, approvals, proactive notifications, Graph.
Slack	Post, channels, threads, history, approvals.
WhatsApp	Templates, media, inbound, provider policies.
Email (SMTP)	Transactional email, templates, attachments, TLS.
Twilio	SMS, WhatsApp, voice, delivery status, numbers.

Credentials are always integration-specific; users must complete OAuth/API key steps in the Integrations Hub before tools appear in DevStudio.

1.34 34. Prebuilt DevStudio tools (family names)

Documentation includes prebuilt templates such as **Gmail**, **Slack**, **Teams**, **Jira**, **MongoDB**, **Custom API**, and related bundles (e.g. Jira/Zendesk patterns). When users reference these, point them to the matching **prebuilt** page for parameter lists and auth scopes.

1.35 35. Long-form FAQs (support)

- **Flows reusable?** Yes — map multiple intents or assistants to the same flow when product logic matches.
 - **Coding required?** Flows are visual; **tools** may require HTTP/SQL/code knowledge depending on integration depth.
 - **Publishing?** Admin/SuperAdmin for production; developers often draft in Dev.
 - **Cross-workspace roles?** Independent per workspace — a user may be Developer in one and Viewer in another.
 - **Tool failures?** Start with exact error string, HTTP status, tool id, environment, and payload redacted of PII.
 - **Intent misfires?** Expand training with real user phrases from logs; check for overlapping intents competing for the same phrases.
 - **Webhook duplicate processing?** Enforce idempotency keys and dedupe tables on the customer side for at-least-once delivery semantics.
 - **Voice issues?** Check telephony metrics, `caller_number` system variable, and provider-specific limits.
-

1.36 36. Workflow creation methods (Flowgen)

Copilot method: Describe the business outcome in natural language (for example: “FAQ assistant that opens Zendesk tickets on escalation”). Copilot proposes a first graph, suggests tools, and may pre-wire DevStudio integrations that already exist in the workspace. Best when speed matters or the team is exploring design. Always review generated prompts for brand tone, compliance, and PII handling before production.

Manual canvas method: Drag blocks, define every variable and edge, and connect tools deliberately. Best when you need deterministic structure, audited branching, heavy variable plumbing, or regulated industries where ML-generated graphs must be inspected line by line.

Choosing: Copilot for bootstrap; manual for precision; hybrid is common — generate then refactor.

1.37 37. Tool creation methods (DevStudio)

Copilot-generated tools: Describe the external system action (“post message to channel X with title and body”). Copilot drafts handler code and parameter schema; developers must still validate auth, timeouts, and error paths.

Manual coding (typically Python): Implement handlers, schemas, and tests directly. Use for sensitive logic, complex transforms, or when corporate standards mandate code review on every change.

Versioning discipline: Draft → test with representative payloads → publish with notes → reference published version from flows. Roll back by pinning flows to earlier tool versions when breaking changes occur.

1.38 38. Intent testing and regression strategy

Tester workflow: Open Intents → Testing → enter phrases that real users say → read predicted intent and confidence.

Refinement loop: Add misses to training data; add contrastive examples if two intents collide; retrain; re-run the same utterances.

Regression sets: Maintain a private list of canonical utterances that must pass before each production deploy; extend the list whenever production misroutes are discovered.

1.39 39. Observability filters (practical)

Dimensions: Time range, assistant, environment, channel, flow/build version.

Use cases: After a deploy, filter to the new version only and compare error rates to the prior window. For cost investigations, filter to the assistant that added a new tool. For webhook issues, filter to environment and narrow the timestamp to the reported event.

Saved filters: Save recurring views (for example “Prod / Customer A assistant / last 7 days”) to speed weekly reviews.

1.40 40. API usage patterns (generic)

Authentication header: Authorization: Bearer YOUR_API_KEY

Webhook POST example (illustrative shape):

POST https://api.example.com/api/triggers/webhook/flow_123

Headers:

Authorization: Bearer YOUR_API_KEY

Content-Type: application/json

Body:

```
{
  "event": "order.created",
  "data": { "orderId": "12345", "customerEmail": "user@example.com" }
}
```

Success response shape (illustrative): JSON containing **status** such as **accepted** and an identifier like **triggerId**.

Safety: Never ship API keys to browsers; rotate keys on schedule; scope keys per workspace; use separate keys per integration if the product allows.

1.41 41. Session log interpretation (support script)

Timeline view: Walk forward in time. Identify the last green step before failure; the failing block name points to Agent vs Tool vs integration.

Decision joints: When the user says “it took the wrong branch,” inspect decision variables — were prerequisites captured earlier in the session? Absent captures often mean the upstream agent did not extract a field — fix prompt or add a dedicated extraction step.

Variable logs: When a downstream tool shows **null** input, trace the variable name backward — often a typo between capture name and tool mapping.

1.42 42. Billing and cost optimization (guidance)

Dashboards: Costs by assistant and environment; token and telephony breakdowns; trends.

Operational practices: Set budgets/alerts; investigate spikes within 24 hours of releases; check whether a new tool issues duplicate calls; evaluate whether a cheaper model tier fits simple classification steps.

Model vs tool spend: Token charges may dominate; telephony may dominate voice stacks — separate these mentally when users say “cost doubled.”

1.43 43. Access, compliance, and audit

Least privilege: Grant Admin only to operators who publish; restrict production keys to CI or senior roles where possible.

Separation: Keep Dev credentials away from Production data sources; some regulated teams use separate workspaces entirely.

Audit: Export logs for external SIEM if required; version notes on builds provide human-readable change history for auditors.

1.44 44. Channel notes (per-channel pitfalls)

Web chat: Often easiest to test; watch for CORS and domain allowlists if the customer embeds the widget.

WhatsApp: Template messages and session windows are policy-sensitive; failed sends often show provider error codes in logs.

Slack / Teams: Permission scopes drive what tools can do; re-auth after scope changes.

Twilio voice: Capture `caller_number` from system variables; watch telephony metrics for dropped calls or STT failures.

Email: Threading and duplicate triggers — idempotency on message ids reduces double-processing.

1.45 45. Data handling and PII (support positioning)

Prompts: Instruct agents not to invent private data; ground on RAG where possible.

Logs: Variable logs may contain user content — treat exports as sensitive.

Tools: Redact PII in ticket updates when posting to third parties; use vault references instead of raw secrets in parameters.

1.46 46. Incident response checklist (platform)

1. Confirm scope — single assistant vs workspace-wide.
2. Capture timestamps and environment.
3. Pull session id from user if available.
4. Filter logs to that session; download timeline.
5. Identify failing block — tool vs model vs channel.

6. If external API — check status pages and credential expiry.
 7. If regression — compare build ids before/after deploy.
 8. Communicate workaround — disable trigger, rollback build, or switch model.
 9. Post-incident — add regression utterance or test case.
-

1.47 47. Advanced intent and trigger scenarios

Overlapping intents: Two intents share phrases — merge intents or sharpen boundaries with negative examples.

Low confidence: Add broader phrasing; check for class imbalance in training lists.

Intent-as-API with wrong channel: User sees API 200 but no Teams message — verify integration funnel and bot installation scope.

Cron storms: Overlapping schedules duplicate work — consolidate schedules or add distributed locks in customer systems.

1.48 48. Build vs flow publishing (terminology)

Flow publish: Saves a versioned flow graph for an environment subject to validation.

Assistant build: Packages assistant-level dependencies — flows, intents, triggers — for a cohesive release. Prefer assistant builds when intents/triggers change alongside flows.

Ordering: When both change, update tools first if interfaces changed, then flows, then intents/triggers, then promote build — minimizes partial states.

1.49 49. Copilot limits (user expectations)

Copilot accelerates drafting but does not replace policy review, security review, or legal review of automated messaging. Users should expect to iterate prompts, validate tools, and run UAT before customer-facing release.

1.50 50. Where to deepen (pointers for chatbot escalation)

- **Legal/compliance:** Customer's counsel — platform docs describe capabilities, not jurisdictional advice.
 - **Vendor outage:** Check third-party status pages when tools fail with 5xx across assistants.
 - **Custom code:** Escalate to customer's engineering when their middleware wraps Phinite APIs.
-

1.51 51. Scenario-based Q&A (support retrieval)

Q: User cannot see the Publish button.

A: Publishing is restricted to Admin/SuperAdmin in the documented matrix. Developers can save drafts. Verify workspace role and target environment; some orgs restrict UAT/Prod publishing to named admins only.

Q: Assistant responds but ignores uploaded PDFs.

A: Confirm RAG data sources are indexed, collections are attached to the **Master Agent** in the Inspector, and the prompt instructs the model to consult knowledge. Empty retrievals often mean the collection has no chunks matching the query language.

Q: Tool returns 401 from Salesforce/Slack/etc.

A: Re-authenticate the integration; tokens expire. Check whether Prod uses a different connected app or OAuth client than Dev. Verify scopes include the attempted operation.

Q: Webhook returns 200 but flow does not run.

A: Distinguish HTTP acceptance from business validation — inspect trigger logs for dropped events, schema mismatches, or mapping to the wrong flow version. Confirm the trigger id in the URL matches the deployed build.

Q: Duplicate tickets created for one email.

A: Email triggers may deliver retries; implement idempotency using provider message ids at the customer automation layer or guard with a short-window dedupe store.

Q: Voice assistant hears wrong words.

A: Review telephony and STT metrics; background noise and codec issues matter. Confirm language/locale settings match the user population.

Q: “Integration limit” during Copilot build.

A: Quickstart references plan limits for multi-tool usage; upgrade path or reduce tools in the first iteration.

Q: Intent API works in DEV, 404 in PROD.

A: Different URLs per environment; confirm build promotion updated intent configuration; verify API path uses PROD workspace identifiers.

Q: Variables empty after tool success.

A: Map outputs to capture variables explicitly; predefined tools may store the whole object under a dotted namespace — reference fields correctly in the next block.

Q: Tester cannot run test conversations.

A: Testers may execute tests while editing is blocked — verify role; some UIs separate “run” vs “edit.”

Q: Need rollback after bad deploy.

A: Use version history — copy prior published version to draft or redeploy prior build per assistant settings; document incident notes in the replacement version.

Q: High token spend overnight.

A: Check for cron jobs with overly verbose prompts, runaway tool loops, or accidental Prod traffic to a Dev assistant through misconfigured channel.

Q: GDPR / data deletion request.

A: Platform capabilities vary — operational guidance is workspace log export and disabling assistants; legal process belongs to the customer’s DPA with their vendor agreements.

Q: Custom API tool times out.

A: Increase timeout if product allows; optimize backend; add retries with idempotency; consider async patterns for long jobs if supported.

Q: Slack messages post to wrong channel.

A: Verify channel id mapping in tool parameters; reconnect integration after workspace migrations; check bot membership in channel.

Q: Google Sheets row append duplicates.

A: Flow may run twice — add conditional checks or use an idempotency column written before append.

Q: Child agent not reusable across flows.

A: Ensure published child flow/agent is referenced correctly; version pins may block updates — bump reference after publish.

Q: Conditional edge never taken.

A: Decision variable may be unset — default handling in prompt or set explicit defaults; log variable values at branch point.

Q: Cannot invite user — email already exists.

A: User may be in another workspace or pending invite — resend or revoke stale invites from Users page.

Q: API key leaked in GitHub.

A: Revoke immediately in workspace; rotate all dependent systems; never commit keys — use CI secrets.

1.52 52. Terminology crosswalk (synonyms users say)

User phrase	Phinite term
Bot / virtual agent	Assistant
Workflow / playbook	Flow
Plugin / connector	Tool / Integration
Skill / utterance routing	Intent
Callback / HTTP start	Webhook trigger
Staging	UAT
Go-live package	Build / published version
Run / conversation	Session
Knowledge base	RAG collections / data sources

1.53 53. Minimum viable triage (ticket fields)

Ask for: **workspace id or name, environment, assistant name, approximate time UTC, session id if visible, channel, build or flow version, screenshot of error (redact secrets), steps to reproduce, expected vs actual behavior.**

1.54 54. Documentation map (full site paths)

Use these slugs when pointing users to the canonical Mintlify pages:

- Getting started: `getting-started/about-phinite`, `getting-started/what-you-can-build`, `getting-started/quickstart`, `workspaces/workspace-overview`
- Users: `user-management/inviting-users`, `user-management/user-management`, `user-management/access-controls`, `user-management/user-roles`
- Assistants: `assistants/overview`, `assistants/types`, `assistants/conversational`, `assistants/email`, `assistants/autonomous`, `assistants/components/*` (flows, intents, tools, builds, triggers, environments, channels)
- Flowgen: `flowgen/overview`, `flowgen/publishing`, `flowgen/methods`, `flowgen/copilot-method`, `flowgen/manual-method`, `flowgen/interface/*`, `flowgen/blocks/*`, `flowgen/connections/*`, `flowgen/agent-block/*`, `flowgen/rag-management/*`, variable topics under `flowgen/`
- DevStudio: `devstudio/overview` and linked tool pages
- Triggers and intents: `triggers-intents/overview`, intents and triggers groups, `webhooks`, `event-triggers`, `mapping`, `intent-as-api`
- Builds: `builds/overview`, `builds/lifecycle`, `builds/publishing`, `builds/configuration`, `builds/environments`
- Channels: `channels/overview`, `channels/supported`, plus channel-specific pages
- Observability: `observability/overview`, `metrics`, `logs`, `filtering`, `billing`

- Support: `support/error-codes`, `support/build-failures`, `support/env-issues`, `support/contact`
 - Integrations Hub: `integrations-hub/overview` and slugs such as `gmail`, `slack`, `microsoft-teams`, `whatsapp`, `telegram`, `email`, `twilio`, `google-bigquery`, `google-sheets`, `google-drive`, `google-calendar`, `mongodb`, `github`, `jira`, `notion`, `salesforce`, `brave-search`, `firecrawl`, `api`, `voice`, `chat-ai`
 - Reference: `reference/faqs`, `reference/glossary`, `reference/api`, `reference/shortcuts`, `reference/release-notes`
-

1.55 55. PDF regeneration (maintainers)

From the docs folder, a typical export matching the bundled PDF is:

```
pandoc support-chatbot-knowledge-rack.md -o support-chatbot-knowledge-rack.pdf --pdf-engine=tectonic
-V geometry:margin=0.72in -V fontsize=10.5pt -V linestretch=1.09 -V documentclass=article --toc
--toc-depth=2 -N
```

Requires `pandoc` and a LaTeX engine such as `tectonic`. Page count varies slightly with fonts and margins; expect roughly eighteen to twenty-two pages.

1.56 56. Document control

Version: Derived from Phinite docs corpus as of 2026-04-04. **Format:** Markdown source `support-chatbot-knowledge-rack.md`; PDF export for offline/RAG chunking. **Maintenance:** Re-run consolidation when major sections ship (new integrations, new auth modes, pricing changes).

End of knowledge rack. For authoritative step-by-step and screenshots, use the full Phinite documentation site. This file is a condensed mirror for RAG and support macros.